

AI Developer Agent

End-to-End Demo Workflow

ADO -> GitHub -> Copilot Agent -> Pipeline -> Deploy

Customer:	Brandsafway
Duration:	45 minutes
Audience:	Engineering leaders, architects, dev leads
Format:	Live demo + narrated walkthrough
Objective:	Show governed AI-assisted delivery

Pre-Demo Setup Checklist

Step A: Environment Readiness

- ☐ Azure DevOps project accessible with Boards view
- ☐ GitHub repo (sautalwar/create_canvas) accessible
- ☐ GitHub Copilot Coding Agent enabled on repo
- ☐ Azure Pipelines connected to the repository
- ☐ Branch protection rules enabled (require PR review + status checks)
- ☐ Pre-created ADO work item template ready as fallback
- ☐ Pre-created GitHub issue draft ready as fallback

Step B: Browser Tabs to Open

- Tab 1: demo-workflow.html overview page
- Tab 2: Azure DevOps Boards / Backlog view
- Tab 3: ADO work item details (pre-created)
- Tab 4: GitHub repo main page
- Tab 5: GitHub Issues tab
- Tab 6: GitHub Pull Requests tab
- Tab 7: Copilot agent issue assignment UI
- Tab 8: Azure Pipelines run results

Step C: Presenter Readiness

- ☐ Signed into both ADO and GitHub
- ☐ Browser zoom increased for readability
- ☐ Notifications and popups disabled
- ☐ Work item wording prepared (health check endpoint)

45-Minute Timing Breakdown

Segment	Time	Notes
Opening + workflow overview	3 min	Use demo-workflow.html
Step 1: ADO work item created	4 min	Define the business request
Step 2: GitHub issue linked to ADO	4 min	Show traceability options
Step 3: Assign Copilot agent	3 min	Show autonomous start
Step 4: Agent creates branch	2 min	Keep this quick
Step 5: Agent develops enhancement	6 min	Biggest value moment
Step 6: Agent commits code + tests	4 min	Emphasize test generation
Step 7: Agent raises PR	4 min	Show AB# linkage
Step 8: Lead review / approve	4 min	Human governance moment
Step 9: ADO pipeline validates	4 min	Existing controls remain
Step 10: Merge + deployment	3 min	Close the loop
Buffer + Q&A	4 min	Delays or deeper questions

Pacing note: If the live agent takes longer than expected, compress Steps 4-7 into a narrated walkthrough using pre-opened tabs and spend more time on review, governance, and Q&A.

Segment 1: ADO Work Item Created

Duration: 4 min **Type:** LIVE DEMO

- [DO]** Create or open a Product Backlog Item in Azure DevOps
- [CLICK]** Boards -> Backlogs -> New Work Item -> Product Backlog Item
- [TYPE]** Title: Add health check endpoint to the API
- [TYPE]** Acceptance criteria: GET /api/health returns HTTP 200 with ApiResponse, includes service status and timestamp, has automated tests
- [EXPECT]** A clearly scoped work item exists with enough detail for GitHub and Copilot to act on
- [SAY]** We start where Brandsafway already plans work: Azure DevOps. The request is intentionally small but real -- production teams need a health endpoint for monitoring, load balancers, and operational readiness.
- VALUE:** Preserves ADO as the planning system of record. Converts a business request into an engineer-ready artifact quickly.

Step F: Fallback Plan

- Option A: Open a pre-created work item and say 'To keep time tight, I prepared this in advance'
- Option B: Show the work item details page screenshot or saved tab
- Option C: Skip live creation, narrate while showing the prepared item

Segment 2: GitHub Issue Linked to ADO

Duration: 4 min **Type:** LIVE DEMO

[DO] Create or open the matching GitHub issue and link it to ADO

[CLICK] GitHub -> Issues -> New issue

[TYPE] Issue title matching the ADO backlog item

[TYPE] Body referencing /api/health endpoint with AB#<id> syntax

[EXPECT] Issue is visibly associated with the ADO item

[SAY] There are three ways to connect ADO and GitHub. The Azure Boards GitHub App is the cleanest default. AB# syntax gives lightweight, bidirectional traceability without changing how developers work.

VALUE: End-to-end traceability from backlog to code. Minimal process change for developers.

Three Integration Approaches

Method	Description
Azure Boards App	Zero-config bidirectional linking (recommended primary)
GitHub Actions API	Custom automation via ado-sync.yml workflow
AB# Syntax	Lightweight linking in commits and PR descriptions

Step F: Fallback Plan

- Option A: Use a pre-created issue already linked to the work item
- Option B: Narrate the linkage using AB# reference and show workflow files

Segment 3: Copilot Agent Assigned Issue

Duration: 3 min **Type:** LIVE DEMO

[DO] Assign the GitHub issue to GitHub Copilot Coding Agent

[CLICK] Issue -> Assignees -> @copilot

[VERIFY] Issue assignee shows Copilot

[SHOW] .github/copilot-instructions.md -- team conventions the agent follows

[EXPECT] Issue is assigned and agent begins processing

[SAY] This is the handoff point. Instead of a human developer pulling the ticket, Copilot becomes the first implementer. It reads the repo, follows local conventions, and works within defined boundaries.

VALUE: Reduces wait time between grooming and implementation. Lets engineers focus on exceptions, architecture, and review.

Step F: Fallback Plan

- Option A: Show a pre-assigned issue
- Option B: Narrate expected actions and move to prepared branch/PR state

Segment 4: Agent Creates Branch

Duration: 2 min **Type:** LIVE DEMO

[DO] Show that Copilot creates a working branch from the issue

[CLICK] Issue development panel / branches view

[EXPECT] New branch appears (copilot/fix-... or issue-based naming)

[SAY] The agent does not work on main. It follows the same branch-based workflow as a human developer. Isolation, auditability, and clean review boundaries remain intact.

VALUE: AI work stays inside normal GitHub controls. Branch protection and PR review requirements remain enforced.

Step F: Fallback Plan

- Option A: Use a pre-created Copilot branch tab
- Option B: Explain automatic branch creation and continue to code diff

Segment 5: Agent Develops Enhancement

Duration: 6 min **Type:** LIVE DEMO

[DO] Walk through the code the agent produced for the health endpoint

[CLICK] Open changed files in the branch or PR diff

[SHOW] `src/routes/index.ts` currently lacks `/health`

[SHOW] Existing pattern in `src/routes/items.ts`

[SHOW] New route following project conventions with `ApiResponse<T>`

[EXPECT] Agent followed repository structure instead of inventing new patterns

[SAY] Copilot is not generating random code. It reads the local architecture and extends it consistently -- Express route modules, `/api/` conventions, strict TypeScript, and the shared `ApiResponse` wrapper.

VALUE: Accelerates common enhancements without bypassing standards. Preserves consistency across APIs.

Step F: Fallback Plan

- Option A: Use pre-opened source files with expected diff
- Option B: Show target files and explain what Copilot adds

Segment 6: Agent Commits Code + Tests

Duration: 4 min **Type:** LIVE DEMO

- [DO]** Show the commits and generated automated tests
- [CLICK]** Commits tab -> files changed -> test file
- [SHOW]** New test file: tests/health.test.ts
- [SHOW]** Test style matching tests/items.test.ts pattern
- [SHOW]** Commit message referencing work item with AB#<id>
- [EXPECT]** Commits show both implementation and test coverage with ADO linkage
- [SAY]** The important point is not just that AI wrote code -- it also wrote tests in the house style. For Brandsafway, that means adoption starts with low-risk changes while reinforcing test discipline and traceability.
- VALUE:** Test-backed changes, not just code generation. Stronger audit trail from work item to commit.

Step F: Fallback Plan

- Option A: Show a prepared commit history or local diff
- Option B: Show tests/items.test.ts as the pattern Copilot follows

Segment 7: Agent Raises PR

Duration: 4 min **Type:** LIVE DEMO

- [DO]** Open the Copilot-created pull request
- [CLICK]** Pull Requests -> open draft PR
- [SHOW]** Draft PR created by Copilot with summary
- [SHOW]** AB#<id> in PR description for ADO traceability
- [SHOW]** Changed files, checks, and review readiness
- [EXPECT]** PR exists with clear traceability and context for review
- [SAY]** Now the work becomes reviewable. Copilot has done the first-pass engineering, but nothing ships automatically. The PR is the control point where humans, policy, and pipeline automation come together.
- VALUE:** Preserves the PR operating model teams already trust. Clean handoff from AI implementation to human governance.

Step F: Fallback Plan

- Option A: Open a pre-created draft PR
- Option B: Narrate expected PR contents using branch diff and issue link

Segment 8: Development Lead Reviews/Approves

Duration: 4 min **Type:** LIVE DEMO

[DO] Review the PR as the development lead and approve it

[CLICK] Files changed -> add comments -> Review changes -> Approve

[SHOW] Review of route design, response structure, test completeness

[SHOW] Optional comment asking for small refinement

[EXPECT] PR receives approval

[SAY] This is the key governance message: AI is not replacing engineering accountability. The lead still reviews architecture, edge cases, and business fit. Copilot speeds up execution, but approval authority remains with the team.

VALUE: Keeps development leads in control. Makes AI adoption compatible with existing approval workflows.

Step F: Fallback Plan

- Option A: Use a prepared approved PR state
- Option B: Verbally summarize checks: route placement, ApiResponse compliance, test coverage, AB# traceability

Segment 9: ADO Pipeline Validates

Duration: 4 min **Type:** LIVE DEMO

[DO] Show PR checks running and Azure Pipeline validating the change

[CLICK] PR checks tab -> Azure Pipelines run -> logs/results

[SHOW] GitHub Actions CI and Azure Pipelines status

[SHOW] Pipeline stages: lint, build, test

[SHOW] Merge blocked until checks pass

[EXPECT] Checks run successfully and PR becomes mergeable

[SAY] Nothing about AI bypasses your enterprise controls. The same pipeline that validates human code validates AI-generated code. For Brandsafway, you can adopt Copilot inside your current governance model.

VALUE: Reuses existing ADO quality gates. Governed AI adoption, not shadow automation.

Step F: Fallback Plan

- Option A: Show a completed successful pipeline run from a prepared tab
- Option B: Open azure-pipelines.yml and explain stages: install, lint, build, test, publish

Segment 10: Merge + Deployment

Duration: 3 min **Type:** LIVE DEMO

[DO] Merge the PR and show the deployment path

[CLICK] Merge pull request -> confirm merge

[SHOW] Merge to main

[SHOW] Deployment stage triggered from Azure Pipelines

[SHOW] Full traceability: work item -> issue -> branch -> commit -> PR -> pipeline -> deploy

[EXPECT] Audience sees merge event and deployment handoff to staging

[SAY] The story ends where delivery teams care most: a change that started as a backlog item is now merged and moving through deployment with full traceability. Faster cycle time without sacrificing review, testing, or operational control.

VALUE: Shorter path from request to value. Clear audit trail. Practical modernization using tools Brandsafway already owns.

Step F: Fallback Plan

- Option A: Show a previously merged PR and completed deployment run
- Option B: Note deploy step is simplified in demo; in production points to real staging target

Competitive Comparison: GitHub + Copilot vs ADO-Only

Capability	ADO-Only	GitHub + ADO + Copilot
Work item planning	***** (native)	***** (ADO stays)
AI code generation	* (limited)	***** (Copilot Agent)
Autonomous issue->PR	N/A	***** (assign @copilot)
Code review workflow	*** (ADO PRs)	***** (GitHub PRs)
CI/CD pipeline	***** (Azure Pipelines)	***** (both work)
Bidirectional tracing	** (manual)	**** (AB# + Boards App)
Test generation	N/A	**** (Copilot writes tests)
Open ecosystem	*** (Microsoft)	***** (GitHub + community)

Hard Questions & Prepared Answers

Step 1: Security

Q: How do we know Copilot is safe to use with our codebase?

A: The safe operating model is the same one you already trust: least-privilege repo access, branch isolation, pull-request review, and pipeline validation. Copilot proposes changes inside GitHub; it does not bypass approvals or push directly to production. For sensitive environments, start with lower-risk repositories first.

Step 2: Cost / ROI

Q: Why pay for Copilot agents if we already have developers and ADO?

A: The ROI case is not headcount replacement; it is cycle-time compression. Copilot removes waiting time on routine tasks, drafts code and tests faster, and lets senior engineers spend more time on architecture, risk, and throughput.

Step 3: Adoption Model

Q: Will developers trust this, or will they fight it?

A: Adoption works best when Copilot starts with boring but important tasks: health endpoints, validation changes, test generation, small API enhancements. Those are low-risk, easy-to-review tasks that build confidence. The lead retains approval authority.

Step 4: Timeline

Q: How long would it take Brandsafway to pilot this?

A: A focused pilot can start in days, not months, if the repo is already in GitHub and basic CI is present. First phase: one or two repositories, a small set of issue types, and a clear review policy.

Step 5: Compliance / Auditability

Q: How does this hold up under compliance and audit requirements?

A: The audit story is a strength: backlog item, linked issue, branch, commits, PR review, status checks, and deployment records are all preserved. AB# references and protected branches create a visible chain of custody from request to release.

Step 6: Existing ADO Workflows

Q: Do we need to abandon Azure DevOps to use GitHub and Copilot?

A: No. ADO remains the planning and work-management layer. GitHub becomes the code, automation, and AI execution layer. Integration is lightweight through Azure Boards app and AB# syntax, or more customized through APIs.

Step 7: Quality Concerns

Q: What if Copilot writes the wrong thing?

A: Then it fails the same way a junior engineer's first draft can fail: in review or in CI. We do not ask Copilot to be infallible; we ask it to produce a faster first draft inside a process that already catches mistakes.

Step 8: Tooling Lock-in

Q: Are we locking ourselves into one workflow or vendor?

A: No single part of this flow is all-or-nothing. The important architectural decision is preserving standard artifacts -- work items, issues, branches, PRs, and pipelines -- so the workflow remains understandable and portable.

Recommended Next Steps

- Run a 2-4 week pilot on one Brandsafway service with low-risk backlog items
 - Define a shortlist of agent-suitable issue types: health endpoints, validation enhancements, CRUD API extensions, test additions
 - Confirm preferred ADO <-> GitHub integration approach (Azure Boards App + AB# recommended)
 - Establish governance baseline: branch protection, required PR approval, required CI + Azure Pipeline checks
 - Measure pilot success: PR cycle time, review time, test pass rate, backlog throughput
 - Expand gradually from one repo to broader engineering workflow once outcomes are proven
-

Closing Statement

"What Brandsafway just saw is not AI replacing delivery discipline. It is AI operating inside delivery discipline: ADO defines the work, GitHub manages the code lifecycle, Copilot accelerates implementation, and your existing leads and pipelines remain the quality gate. That is the model I would recommend for a practical, low-risk pilot.